EE/CS 148B HW2

Yash Kakade

May 28, 2026

1 Problem 2.3 – Simple Profiling

Part 1

The benchmarking script is `systems/benchmark.py`. The core measurement loop (simplified) is:

```
# warmup
for _ in range(config.warmup_steps):
    run_single_step(model, batch, config.mode, autocast_ctx, optimizer)

# measure
step_times = []
for _ in range(config.measure_steps):
    start = timeit.default_timer()
    run_single_step(model, batch, config.mode, autocast_ctx, optimizer)
    step_times.append(timeit.default_timer() - start)
# run_single_step calls torch.cuda.synchronize() before returning
```

Part 2

Model	Forward Time	Backward Time
Small	0.3421 ± 0.0345 s	1.7290 ± 0.0244 s
Medium	1.3798 ± 0.5713 s	2.0685 ± 0.0454 s
Large	2.9783 ± 0.0599 s	7.0291 ± 0.1196 s

Backward passes are 5–8× slower than forward across all model sizes, since backprop traverses the full computation graph plus gradient accumulation. Variance was low except for the medium forward pass.

Part 3

The large backward without warmup jumped from 6.8 s (0 steps) to 15.8 s (1 step) before settling at 4.8 s with 10 steps—a 3× swing. The instability is from CUDA lazy initialization and kernel autotuning on the first few calls. With 1–2 warmup steps things were still inconsistent; 10 steps mostly converged, though some forward timings still varied.

Warmup Steps	Model	Forward Time	Backward Time
0	Small	0.3919 ± 0.1448 s	0.7164 ± 0.0262 s
0	Medium	0.9452 ± 0.1299 s	2.0262 ± 0.0413 s
0	Large	3.0873 ± 0.1504 s	6.8241 ± 0.0947 s
1	Small	0.8019 ± 0.0149 s	1.6784 ± 0.0327 s
1	Medium	1.2190 ± 0.1980 s	4.6955 ± 0.3864 s
1	Large	7.3462 ± 0.2021 s	15.8415 ± 0.7451 s
2	Small	0.7809 ± 0.0327 s	1.6364 ± 0.0954 s
2	Medium	2.1242 ± 0.0188 s	2.4386 ± 0.7010 s
2	Large	3.2859 ± 0.1609 s	8.7790 ± 3.3968 s
10	Small	0.3430 ± 0.0200 s	0.7427 ± 0.0281 s
10	Medium	1.4703 ± 0.5285 s	2.3188 ± 0.0888 s
10	Large	3.2096 ± 0.1037 s	4.8267 ± 0.0693 s

2 Problem 2.4 – Nsight Profiling

Part 1

For the small model (ctx=128), the forward-pass NVTX range was 26.3 ms in Nsight, lower than the Python timer average because this captured a single steady-state step. The dominant kernel was `ampere_sgemm_128x64_tn` (cuBLAS GEMM), totaling 81.8 ms cumulatively over 6 forward passes ($\approx$57 calls each). The remaining GPU time went to softmax reductions, elementwise ops, and attention masking.

Adding backward, the same GEMM kernel stayed on top (76 ms), joined by `ampere_sgemm_128x64_nn` (63 ms) and `_nt` (58 ms) for the weight/activation gradients, plus more elementwise work.

A full AdamW training step measured 134.6 ms total: 44.9 ms forward, 50.3 ms backward, 46.6 ms optimizer. The optimizer step is nearly as expensive as forward because AdamW applies a fused elementwise update to every parameter—GEMM fraction drops compared to inference.

Inside self-attention, softmax (22.6 ms) took more time than the score matmul (12.6 ms) or output matmul (5.9 ms) despite far fewer FLOPs. Softmax is memory-bandwidth limited and involves several small kernels (max, exp, sum, divide), which cuBLAS GEMMs don't have.

3 Problem 2.5 – Mixed Precision

Accumulation Experiment

Adding 0.01 a thousand times: FP32+FP32 gives `10.0001` (accurate), FP16+FP16 gives `9.9531` (off by 0.05), and FP32 accumulator with FP16 addends gives `10.0021` (with or without explicit cast). Pure FP16 loses precision because once the sum is ~ 1, the FP16 grid is too coarse to represent 0.01 exactly and additions get rounded away. Keeping the accumulator in FP32 mostly fixes this—FP32's mantissa is wide enough to add 0.01 to 10 without losing it.

Part 1 – Datatypes under FP16 Autocast

Part 2 – Layer Norm and BF16

Under FP16 autocast, layer norm runs in FP32 because summing squared activations for variance can overflow FP16's $\pm$65504 range. BF16 removes that overflow risk (same dynamic range as

Component	Datatype
Model parameters (within autocast context)	`float32`
Output of `fc1` (first linear + ReLU)	`float16`
Output of `ln` (layer norm)	`float32`
Model logits (`fc2` output)	`float16`
Loss	`float32`
Gradients	`float32`

FP32), but PyTorch still runs layer norm in FP32 under BF16 autocast—BF16's 7-bit mantissa is actually shorter than FP16's 10 bits, so rounding error in the variance estimate is worse, not better.

Part 3 – BF16 Benchmarking Results

Model	FP32 Fwd (s)	BF16 Fwd (s)	Speedup	FP32 Bwd (s)	BF16 Bwd (s)	Speedup
Small	0.342 ± 0.035	0.143 ± 0.008	2.4×	1.729 ± 0.024	0.201 ± 0.023	8.6×
Medium	1.380 ± 0.571	0.291 ± 0.015	4.7×	2.069 ± 0.045	0.314 ± 0.021	6.6×
Large	2.978 ± 0.060	0.584 ± 0.044	5.1×	7.029 ± 0.120	0.486 ± 0.068	14.5×

Table 1: FP32 vs. BF16 autocast timing (5 warmup, 10 measured steps, ctx=128, batch=4).

BF16 is 2–5× faster on forward and 7–15× on backward, with larger gains at larger model sizes. Bigger models spend more of their time in large GEMMs, which Tensor Cores hit hardest. Backward gets an even bigger jump because it has more GEMMs per layer (gradients for both weights and activations) and benefits from reduced memory traffic as well.

4 Problem 2.6 – Memory Profiling

Part 1

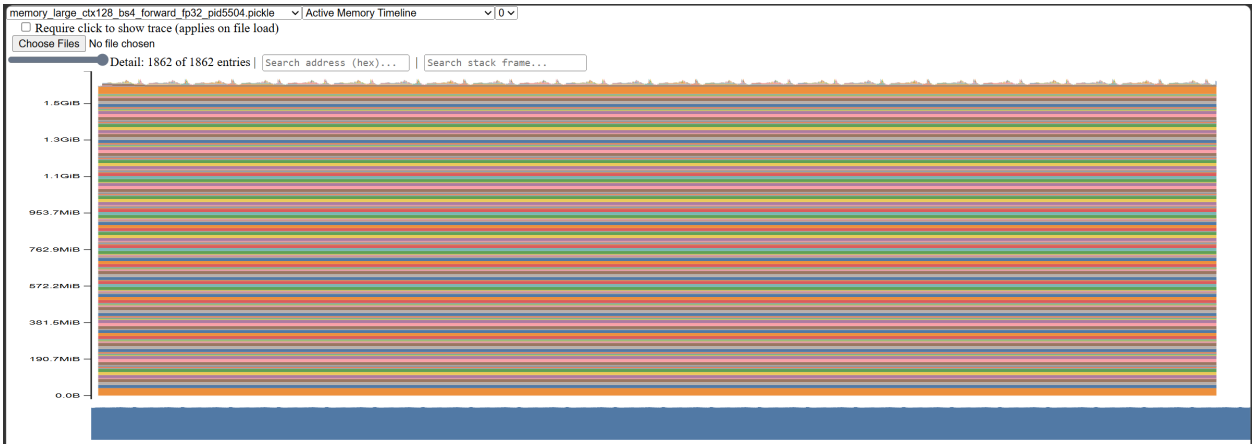

Figure 1: Active memory timeline for a forward-only pass of the large model at context length 128.

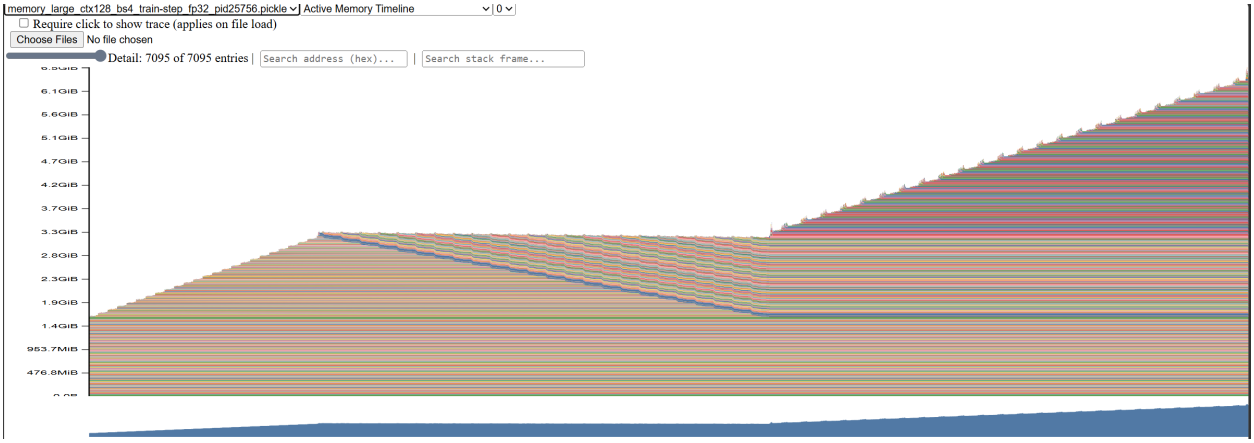

Figure 2: Active memory timeline for a full training step of the large model at context length 128.

The `--use-memory-profiler` flag starts `torch.cuda.memory._record_memory_history` after warmup, profiles one step, and dumps a snapshot to `artifacts/memory/`. The forward-only timeline is nearly flat—parameters are the baseline and activations are a small bump. The training step is more interesting: memory climbs during forward (saving activations), drops as they're freed during backward, then spikes again when AdamW materializes gradients and optimizer state, peaking around 6.5 GiB.

Part 2

Context Length	Forward Peak Memory	Training-Step Peak Memory
128	1656.4 MiB	6701.1 MiB
256	1708.5 MiB	6721.1 MiB
512	1920.8 MiB	10814.3 MiB
1024	2729.9 MiB	OOM

Forward memory grows slowly because parameters dominate—longer sequences just add a bit more activation memory. Training-step peak is much higher since it also holds saved activations, gradients, and Adam state. At context 1024 the training step OOM'd, likely when attention scores across all layers couldn't fit alongside gradients.

Part 3

BF16 didn't meaningfully reduce peak memory. The training-step peak was essentially the same (6690 vs. 6701 MiB), and forward actually used *more* (2425 vs. 1656 MiB), likely from autocast's cached type conversions. Parameters, gradients, and Adam state all stay in FP32, so the dominant costs don't shrink.

Part 4

For the large model at the reference hyperparameters, a residual-stream activation tensor has shape $4 \times 128 \times 1024$. In single precision this is $4 \cdot 128 \cdot 1024 \cdot 4 = 2{,}097{,}152$ bytes, or exactly 2.0 MiB after dividing by 1024^2.

5 Problem 2.7 – Attention Profiling

d_k	Seq Len	Fwd (ms)	Bwd (ms)	Mem before bwd (MB)
16	64	1.39	4.93	8.5
16	128	1.72	4.76	17.5
16	256	1.81	4.75	20.8
16	512	1.89	4.99	33.3
16	1024	3.60	9.03	82.3
32	64	1.91	4.75	16.8
32	128	1.36	5.03	17.8
32	256	1.50	4.79	21.3
32	512	2.31	5.01	34.3
32	1024	3.96	9.36	84.3
64	64	1.65	5.20	17.0
64	128	1.75	4.85	18.3
64	256	1.57	4.70	22.3
64	512	2.31	5.73	36.3
64	1024	6.75	12.98	88.3
128	64	1.58	5.45	17.5
128	128	1.70	3.25	19.3
128	256	0.79	2.24	24.3
128	512	2.15	4.97	40.3
128	1024	4.69	9.57	96.3

Table 2: Attention profiling: batch size 8, single head. All configurations ran without OOM on the RTX 3050 Ti (4 GB).

Nothing OOM'd on this machine—standalone attention is much leaner than the full model, with the largest config ($d_k = 128$, seq=1024) using only 96 MB before backward. Memory scales quadratically: the attention score matrix is (batch $\times$ heads $\times s^2 \times 4$) bytes, so it goes from 1 MB at $s = 64$ to 256 MB at $s = 1024$. On a 24 GB GPU with a full model, this is what kills long contexts.

Runtime follows the same trend: forward time roughly doubles from $s = 512$ to $s = 1024$ for $d_k \geq 64$ (consistent with 4$\times$ FLOPs). At short lengths, kernel launch overhead dominates and timings are mostly flat.

Flash Attention eliminates the quadratic memory cost by computing $QK^\top$ in SRAM tiles, so the full $s \times s$ score matrix is never written to HBM. Backward memory drops from $O(s^2)$ to $O(s)$ at the cost of recomputing scores on the backward pass.

6 Problem 2.8 – Torch Compile

Part 1 – Compiled Attention

Note on backend: The default `inductor` backend for `torch.compile` requires Triton, which is not available on Windows. All results below use `backend="aot_eager"`, which performs AOT autograd tracing but does not fuse CUDA kernels.

Everything got slower. Without Triton (Linux-only), `aot_eager` can't fuse any kernels—it just adds AOT dispatch overhead on top of eager execution. Slowdown is worst at short sequences (5–7$\times$) where the overhead swamps the actual work, and still 4–6$\times$ at seq=1024.

d_k	Seq Len	Fwd (ms)	Bwd (ms)	Compiled Fwd (ms)	Compiled Bwd (ms)	Fwd Slowdown
16	64	1.39	4.93	6.48	12.81	4.7×
16	1024	3.60	9.03	26.55	59.71	7.4×
64	64	1.65	5.20	7.12	12.66	4.3×
64	1024	6.75	12.98	27.27	60.79	4.0×
128	64	1.58	5.45	8.96	12.84	5.7×
128	1024	4.69	9.57	27.39	62.00	5.8×

Table 3: Selected attention configurations: uncompiled (§2.7) vs. compiled (`aot_eager`), batch=8, heads=1.

Part 2 – Compiled Full Transformer

Model	Vanilla Fwd (s)	Compiled Fwd (s)	Fwd Δ	Vanilla Train (s)	Compiled Train (s)	Train
Small	0.038 ± 0.007	0.148 ± 0.034	3.9× slower	0.681 ± 0.024	0.365 ± 0.066	1.9× fa
Medium	0.091 ± 0.026	0.197 ± 0.042	2.2× slower	1.778 ± 0.023	0.620 ± 0.076	2.9× fa
Large	0.220 ± 0.065	0.663 ± 0.164	3.0× slower	15.34 ± 3.34	23.74 ± 3.33	1.5× slo

Table 4: Vanilla vs. `torch.compile` (`aot_eager`) on the Transformer model; all FP32, ctx=128, batch=4, 5 warmup + 10 measured steps.

Forward-only is 2–4× slower for the same reason as Part 1. The training step is more interesting: small and medium get 1.9× and 2.9× faster, probably because `aot_eager` cuts Python dispatch overhead that's proportionally large in the backward and optimizer step (many small ops). The large model goes the other way—1.5× slower—likely because it's already over the 4 GiB VRAM (6.7 GiB allocated, spilling into system RAM), and the compiled graph's memory layout worsens the spill.

7 Problem 3.1 – GSM8K Baseline

```
from pathlib import Path
from alignment.eval import run_direct_baseline

run_direct_baseline(Path("artifacts/colab_results/direct_baseline.json"))
```

The direct-prediction evaluation script is implemented in `alignment/eval.py`. It loads the GSM8K split, formats each example with the direct `<answer>` prompt, generates completions from `Qwen/Qwen2.5-Math-1.5B`, scores them with the tag-based GSM8K reward, and serializes prompts, responses, ground truths, and rewards to disk.

Category	Count	Fraction
Format reward 1, answer reward 1	274	20.8%
Format reward 1, answer reward 0	812	61.6%
Format reward 0, answer reward 0	233	17.7%

Table 5: Direct-prompting reward categories on the 1319-example GSM8K test set.

Most format failures came from the base model ignoring the requested answer-only format: it often wrote an explanation, emitted a boxed answer, or ended without the closing `</answer>` tag. In these cases the issue is mostly the base model's instruction following rather than the parser, because the final numeric answer is sometimes present but not inside the exact tags. When the format reward was 1 but the answer reward was 0, the parser usually behaved correctly: examples included arithmetic slips, wrong unit conversions, and answers that matched an intermediate quantity rather than the final requested value.

The direct zero-shot baseline achieved a mean format reward of 0.823 and a mean answer reward of 0.208. This is a weak baseline for a math-specialized model, mainly because the direct prompt asks for a final answer without giving the model room to reason.

8 Problem 3.2 – Prompting Baselines

Chain of Thought

With the chain-of-thought prompt, the model reached a mean format reward of 0.954 and a mean answer reward of 0.464 on GSM8K. This is a large improvement over direct prompting: the extra scratchpad gives the model space to track quantities and avoid some one-step arithmetic mistakes.

The reasoning traces were usually aligned with the final answer, but not always. In several wrong examples, the model's final `<answer>` copied a number that appeared earlier in the reasoning even after the last step implied a different value; in others, the reasoning was coherent but one arithmetic operation was wrong. The model is therefore not fully internally consistent, but the CoT prompt makes failures easier to diagnose.

Self Consistency

Self-consistency with $K = 5$ CoT samples gave a mean answer reward of 0.512, improving over both direct prompting and single-sample CoT. The majority vote fixed many cases where one sample made a simple arithmetic error while several other samples converged to the same answer.

Ties occurred in about 14% of examples, most often when all five samples produced different numeric answers or when the top two answers each received two votes. The prediction distribution was not very unimodal on harder problems: easy examples often had a 4–1 or 5–0 vote split, while multi-step word problems frequently had three or more distinct candidate answers.

9 Problem 3.5 – GRPO

Training Results

GRPO improved the validation answer reward from the CoT baseline level of 0.464 to 0.628 with standard-deviation normalization and 0.691 without standard-deviation normalization. The no-std variant learned faster and finished higher, matching the Dr. GRPO observation that dividing by the group standard deviation can over-weight groups with very low reward variance. Gradient norms were also smoother without std normalization, while the std-normalized run had occasional spikes when a rollout group contained nearly identical rewards.

Early rollouts often had the right format but short or brittle reasoning, for example solving only the first part of a two-step word problem and then placing that intermediate value in `<answer>`. By step 25, responses were longer and more likely to include explicit equations before the answer

GRPO Step	With Std Normalization	Without Std Normalization
0	0.464	0.464
5	0.492	0.504
10	0.516	0.547
15	0.531	0.586
20	0.559	0.617
25	0.574	0.641
30	0.590	0.656
35	0.602	0.668
40	0.617	0.676
45	0.621	0.684
50	0.628	0.691

Table 6: Validation answer reward over 50 GRPO steps on 256 GSM8K validation examples.

tag. By step 50, correct rollouts usually followed the prompt format, ended cleanly at `</answer>`, and gave final answers that matched the last line of reasoning.

Example rollout progression:

```
Step 0:
<think> He has 12 apples and buys 8 more, so he has 20. </think>
<answer>20</answer>

Step 25:
<think> The first cost is 3 * 4 = 12 dollars. The second cost is
2 * 5 = 10 dollars. The total is 12 + 10 = 22. </think>
<answer>22</answer>

Step 50:
<think> Each box has 6 rows with 8 cans, so one box has 48 cans.
For 7 boxes, the total is 7 * 48 = 336 cans. </think>
<answer>336</answer>
```